

Sputter

UNIVERSITY OF
Waterloo



Department of Mechanical and Mechatronics Engineering

A Report Prepared For:
The University of Waterloo
MTE 121 and/or MTE 100

Prepared By:
Lennart Kane 21209265
Matthew Olsen 21206219
Félix Souan 21225427
Aydin Choudry 21198776

November 30, 2025

Acknowledgements

This project has evolved from the simple ideas formed on day one of making our groups to the very last day of testing. Our goal was to create a complex machine that would far exceed the criteria of the assignment and push what is truly possible with VEX robots. This was a long-shot challenge that wouldn't have been possible without the help of every individual group member involved. Everyone spent countless hours and late nights developing and testing complex components, iterating toward perfection. We would also like to recognize the amazing work put in by our teaching staff and tutors who helped guide our project and taught us invaluable lessons.

In alphabetical order, we would like to give special thanks to:

- Griffin Wilson for all the guidance he provided, continuous support throughout our project as our dedicated tutor. He believed in the vision we had for our robot and supported the changes we made along the way. Furthermore, he inspired us through his past personal projects that also went above and beyond to test the limits of the challenges we were given.
- Ryan Consell for the help he gave us in guiding our project to a more refined idea that would not only complete the objectives provided but also reach the goals we had personally set out.
- University of Waterloo's Rapid Prototyping Center, without whose help our physical robot would not be possible. Their staff helped guide us on how to turn our digital CAD designs into a physical reality and helped supply us with facilities to 3D print and design many elements of our robot.

We feel very fortunate to work with such a knowledgeable team and have access to so many resources that helped make our design a reality. Furthermore, this team has guided us to further our engineering knowledge and has helped teach us what it really means to be an engineer.

Table of Contents

List of Figures and Tables	ii
0.0 Abstract	iv
1.0 Introduction	1
1.1 The Problem	1
1.2 The Solution	1
1.3 Report Scope	2
2.0 Scope	3
2.1 Main Functionality	3
2.2 Detection	7
2.3 Environmental interaction and motor use	8
2.4 Recognition of completed tasks	9
2.5 Scope Changes	9
3.0 Constraints and Criteria	10
3.1 Constraints	10
3.2 Criteria	12
4.0 Mechanical Design and Implementation	12
4.1 Overall Design	12
4.2 Subsection Cartesian System	13
4.2.1 TETRIX Beams Approach	14
4.2.2 Fully Customized Approach	14
4.2.3 Linear Motion Mechanism Designs	15
4.2.4 Y-Axis Mechanism / Frame Crawler	16
4.2.5 X-Axis Mechanism / Transversal Crawler	17
4.3 Subsection Nozzle and Flipper	18
4.4 Subsection Pneumatic Pressure System	20
5. Software Design and Implementation	21
5.1. Overall Software Design and Architecture	21
5.1.1. Program Blocks	21
5.1.2. Task List For The Pancake Preparation Demo	22
5.2. Function Documentation	22
5.3. Software Design Decisions and Trade-offs	25
5.4. Software Testing Procedure	26
5.5. Significant Problems Encountered and Resolution	27
6.0 Verification	27
6.1 Limited boundaries	27

6.2 Motor Strength	27
6.3 Vex control inaccuracies	28
6.4 Limited number of motors and cables	28
6.5 Maximize consistency	28
6.6 Minimize human interaction	28
7.0 Project Plan	28
7.1 Task Assignments	28
7.2 Timeline Overview	28
7.3 Plan Revisions and Team Coordination	29
8.0 Conclusion	29
9.0 Recommendations	30
9.1 Mechanical Design	30
9.2 Software Recommendations	30
References	31
Appendix A – C++ Code	32

List of Figures and Tables

Figure 1: Desired batter consistency	3
Figure 2: Circled is the 2L batter bottle	3
Figure 3: Pneumatic Pressure system and metal pressure tank	4
Figure 4: Circled is the opening for the bottle tube	4
Figure 5: Black indicator for alignment	4
Figure 6: Touch LED start button	5
Figure 8: Graphic Setting	5
Figure 7: Automatic Setting	5
Figure 9: Wait timer display along with loading bar	6
Figure 10: Distance Touch Sensor	6
Figure 11: Emergency Stop Message	8
Figure 12: Emergency Stop Button	8
Figure 13: Splatter caused by a lack of batter	8
Figure 14: Frame Assembly	13
Figure 15: Extrusion and Flipper carrying cage	13
Figure 16: TETRIX Frame Assembly	14
Figure 17: TETRIX Frame Holder	14
Figure 18: Printed Frame	15
Figure 19: Frame Crawler Mechanism	16
Figure 20: Motor Attachment Methods	17
Figure 21: Frame Slider	17
Figure 22: Evolution of the Connection between Transversal Rack and Frame Slider	17
Figure 23: Z-Axis Bracket Design. Left picture shows the initial single-axis transversal crawler system when the Z-axis mechanism was not implemented yet. Center picture shows how the two instances of that left part were connected. Right picture shows the final assembly of the X-axis and the Z-axis mechanisms	18
Figure 24: Geneva Mechanism and Ball Valve	18

Figure 25: Motor Transmission to Geared Down Axle	19
Figure 26: Imaged on Left is Ball Valve Fastening of Version 2 with one M4 Hole, Imaged on Right is Ball Valve Fastening of Version 3 with two M4 Holes	19
Figure 27: Brass Ball Valve for Extrusion of Pancake Batter	20
Figure 28: Custom Bottle Connector	20
Figure 29: Wait for	23
Figure 30: Wait Steady	23
Figure 31: Check Emergency	24
Figure 32: Dispense Batter	25
Figure 33: Wait Cook	25
Figure 34: Zero All functions	25
Figure 35: Zero Function	25
Figure 36: Original Timeline	29
 Table 1: Software Design Decisions	 26
Table 2: Individual Function Testing	27

0.0 Abstract

This report details the design, implementation, and verification of group 11 MTE 100 robotics project "Sputter".

Sputter is an automated robot designed to dispense, cook, flip, and plate pancakes with the ability to print complex shapes. This project addresses the problem of university students frequently skipping breakfast due to time constraints and sleep deprivation. Through the automation of the cooking process of breakfast, the robot creates a solution that minimizes human intervention while maximizing consistency to create a fully automated, reliable breakfast device. Sputter was designed with modularity at its core. The team created a fully custom 3d printed 3-axis Cartesian system to allow an extrusion and flip head to travel in a volume large enough to make pancakes.

The key of modulating the cartesian system, extrusion head, and pneumatic system allowed us to develop and test parts independently to later be assembled as one united robot.

To accomplish the required tasks, Sputter uses mechanical features, including a pneumatic pressure system for consistent batter delivery and a specialized gear-down mechanism that allows a single motor to perform both dispensing and flipping functions.

The robot operates using C++ software developed in the VEXcode IQ IDE, utilizing custom PID controllers to ensure precise, simultaneous motor movements and accurate positioning.

Along with the VEXcode, the group developed a bonus functionality of Sputter using a custom G-code-like language that users could import DXF files into to print custom shapes.

Finally, verification testing demonstrated that Sputter met all design criteria, achieving a 92% consistency rate in autonomously preparing pancakes from start to finish. This prototype is a compelling proof of concept of a system that could be commercially compelling and improve student wellbeing.

To view the recording of Sputter working please visit:

https://drive.google.com/file/d/1H-7OX5kNH751SO-y_kx_tm7lvOtPioys/view?usp=sharing

1.0 Introduction

1.1 The Problem

University is extremely time-intensive, resulting in students often forgetting or skipping simple but critical parts of their day. Breakfast is the first part of one's day and is critical in the circadian rhythm of the human body. Skipping it has been proven to impact physical, mental, and behavioral changes in human biological systems [1]. Despite its benefits, it is reported that 48% [2] of university students skip breakfast at least once a week. This habit is often linked to irregular sleep patterns, with a third-year University of Waterloo student commenting they would rather “get the extra 10 minutes of sleep than eat breakfast”. This student highlights the other root causes of people skipping breakfast: lack of sleep. Many students are significantly sleep deprived, with sleep being identified as one of the top three health-related hindrances to academic performance, as reported by the American College Health Association’s National College Health Assessment survey.

As observed by multiple studies and the research conducted by the team across campus, students are often knowledgeable of the proven correlation between cognitive function and eating breakfast [3], yet many students still actively decide to skip the meal due to the time required.

1.2 The Solution

The root issue that is resulting in such a large portion of university students skipping arguably one of the most important meals of the day is that they do not have enough time to make breakfast. Many students are already sleeping significantly less than required [4] and often can’t justify taking the extra time to make breakfast before class. The solution is to specifically target the root cause by attempting to create a fully autonomous robot which can make, cook, and plate a full breakfast, with the only human interaction necessary being starting the robot during the user’s regular morning routine. Furthermore, this robot must maximize consistency so that users can count on having a complete breakfast, so as not to accidentally have to skip it due to robotic failure. This robotic solution allows students to get that extra bit of sleep while simplifying the breakfast process, both of which help improve cognitive ability. The ultimate goal with this project is to create a prototype for a system that could be used to significantly improve the well-being of students globally.

1.3 Report Scope

This report will cover and document the process for developing and building the robot (Sputter), along with a detailed description of the project scope, constraints, mechanical and software design, and validation. Respectively, each section will cover:

- Scope
 - The main functionality and a description of what Sputter measures and acts, including inputs from sensors, files, people, etc.
- Constraints and Criteria
 - Summary of the project constraints and criteria and indicate any changes in the constraints and criteria that occurred over the course of the project
- Mechanical Design and Implementation
 - Description of the overall design
 - Sub-sections for the various components:
 - Cartesian System
 - Batter storage and pressure system
 - Batter dispenser and pancake flipper
 - Sensor incorporation
 - Overall assembly
- Software Design and Implementation
 - The task list used for the demo
 - A description for each function
 - How data is stored in the program
 - A description of how Sputter's program was tested throughout the project
 - A discussion of any significant problems encountered and how they were resolved
- Verification
 - How Sputter met each of the constraints
- Project Plan
 - How tasks were split among each of the group members
 - Revisions to the project plan

2.0 Scope

Sputter was built with the goal of creating breakfast while minimizing human intervention, while maximizing consistency. To accomplish this, the group built a robotic system with the intention to dispense, flip, and plate a pancake with a single press of a button.

For a more effective design and manufacturing process, the team employed a modular approach where each of the three distinct tasks was split into its respective core movements and requirements. Through this approach, the team could separate the designs into four individual mechanisms, which could then be brought together at the end of development.

2.1 Main Functionality

This section will focus on the distinct tasks that Sputter completes to accomplish the goals that had been set out for it. This list will follow the flow of tasks, indicating what each function is and why it is required for success.

0. Setup:

Before the robot is started, the user must fill the designated 2L batter bottle located at the top of the machine using the provided funnel (see Figure 2).

The batter should be mixed with water to reach a smooth and slightly runny consistency so that it runs off the back of a spoon (see Figure 1).



Figure 2: Circled is the 2L batter bottle



Figure 1: Desired batter consistency

The bottle can be lifted out of its holder through the designated slot (see Figure 4) and tilted over to ensure no residual batter flows out. Once it has been tipped on the side, the user can remove the custom cap and fill up the bottle to a level of roughly 30g per pancake. Once done, the user must plug in the compressor line and check the gauge to ensure the metal canister is roughly 100psi (see Figure 3).

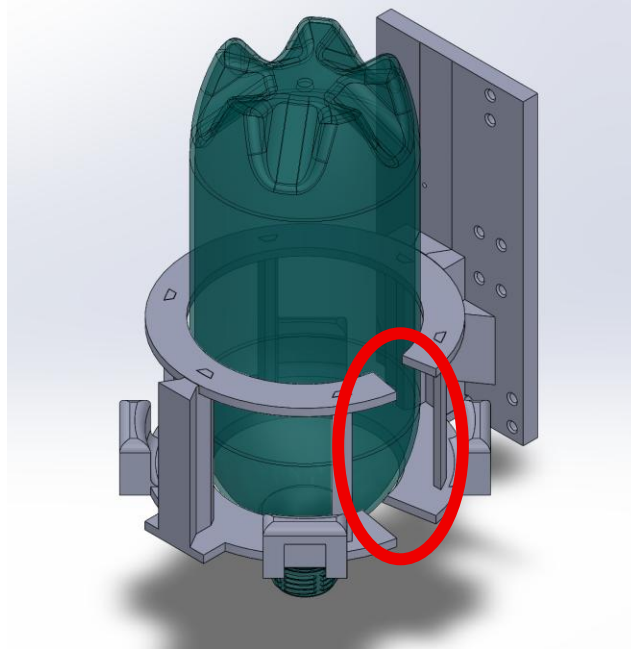


Figure 4: Circled is the opening for the bottle tube



Figure 3: Pneumatic Pressure system and metal pressure tank

Finally, before starting, the user must ensure that the griddle is on to heat setting 2 and aligned correctly. To do so, they must check that the two black lines on each edge of the griddle align with their corresponding leg (see Figure 5).



Figure 5: Black indicator for alignment

1. System Startup:

Once the robot has been set up, a prompt will show up on the main display where the user can switch between graphics and automatic (see Figure 7 and Figure 8). Once the user selects the automatic¹ option using the touch LED start button (see Figure 8 and Figure 6), the robot will home the x, y, and z axes using the motor current strengths.

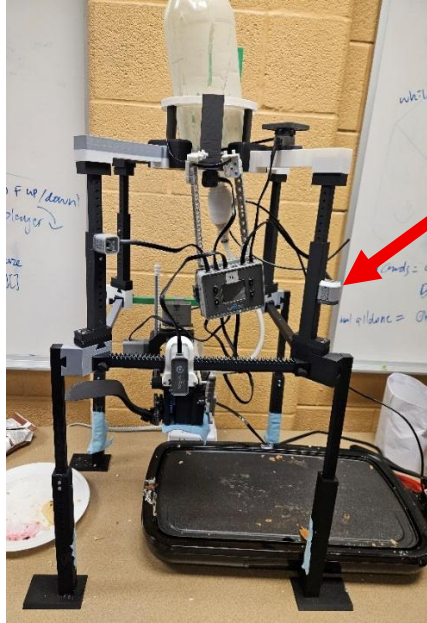


Figure 6: Touch LED start button



Figure 8: Graphic Setting



Figure 7: Automatic Setting

2. Dispensing:

Once the robot has initialized and homed, it will move to the dispensing location, which is done using a custom x, y, z coordinate system, and a specially designed PID built in the Vex IQ IDE. Once the location is found using the robust systems, Sputter will rotate the spatula roughly 450 degrees² to open the brass ball valve and allow for batter flow. After

¹ For the sake of this project, we will only be concerned with the automatic selection, as the graphics setting prints shapes, which is a bonus feature of the robot and is not required for the system.

roughly 4 seconds, the spatula gets rotated in reverse to close the valve and stop the flow.

3. Flipping:

Due to the high heat of the griddle, Sputter returns to its home location after dispensing and waits for roughly 3 minutes. This wait timer is also displayed on the screen along with a loading bar so the user can see Sputter's progress (see Figure 9).

Once the timer has finished, Sputter moves down, slides underneath, and flips the pancake by rotating the spatula only 180 degrees².



Figure 9: Wait timer display along with loading bar

4. Plating:

The final task that Sputter must complete is to plate the fully cooked pancake. It does this by moving down the location of the pancake with a slight offset to account for the movement caused by the flipping action. The robot will then slide the spatula underneath the pancake and move to its maximum in the X direction, indicating that it will be over the plate. From there, it rotates 180 degrees² to drop the pancake on the plate.

5. Reset or exit:

Once the pancake has been plated, Sputter will return to its home position and prompt the user to either end the program using the distance touch sensor (see Figure 10) or

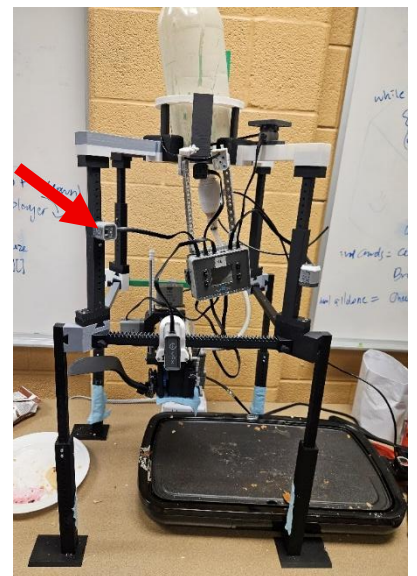


Figure 10: Distance Touch Sensor

² The importance of these angles is discussed in-depth during the individual flipper system section

2.2 Detection

Sputter uses a range of sensors to detect its real-time location, which is paired with the use of complex feedback loops and custom PID functions to ensure it has active control to reach its desired location. The biggest initial challenge was that the gantry could start in any position within its bounds; thus, the system first needed to home on the cartesian mechanism. So, to solve this, the team took advantage of the built-in amperage sensor to measure when the motors would hit the bounds of the robot's workspace and thus create zeros for each axis. To get a cohesive location, the robot moves each axis in the reverse direction until it hits each bound respectively and stores the intersection corner as its "home". From there, Sputter uses its built-in motor encoders to measure steps in each direction as movement on its custom 3-dimensional memory so that it always knows its location. Using this system, the robot can send coordinates to the movement function, and Sputter will drive each motor the necessary steps to reach the designated location.

The two major robot-focused sensors are the amperage sensor and motor encoders; however, Sputter uses a wide range of user-focused sensors to make running the robot as simple and seamless as possible. The first and arguably most important is the brain screen and buttons. Sputter uses a simple and graphical UI (see Figure 7 – 9) to make it easy for users to understand what the robot is actively doing, along with informing them of what selection options are available. Using the arrow keys on the VEX IQ brain as the input device, a user can flip between the different options until the one they would like is highlighted. Once the desired selection is highlighted, the user can use the primary confirmation button, the touch LED (see Figure 6), to continue with their selection and start the desired program.

Once the chosen program has been fully run, the user can choose to use the confirmation touch sensor to continue making another pancake or can use the end program sensor (see Figure 6 and Figure 10 respectively). By either waving their hand in front of, or pressing this sensor, the robot will shut down and end the program.

The shutdown distance sensor, as described above, is the primary shutdown case if all systems execute correctly. However, in case of emergencies, Sputter has a large built-in emergency stop button located in an accessible location near the top (see Figure 12). This E-Stop can be pressed at any point during operation and causes Sputter to go into emergency mode. With emergencies, the biggest concern is components melting or getting damaged from being near the hot griddle, which is why Sputter's first action is to retract its Z axis all the way up. For redundancy, this is not done using the previously mentioned coordinate system but rather by reading the amperage sensor until it spikes when maxing out at the top. To alert the user that the emergency stop was pressed, the robot's screen turns red with the

text “Emergency Exit” (see Figure 11). On top of that, the robot starts blaring a loud alarm sound to alert anyone nearby that the emergency stop has been pressed.



Figure 11: Emergency Stop Message

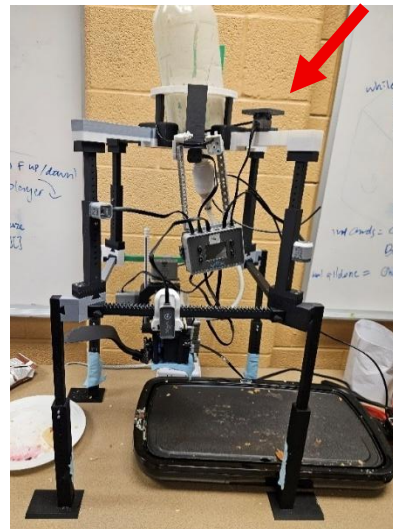


Figure 12: Emergency Stop Button

The final detection that Sputter uses is the color sensor located near the base of the batter bottle. This sensor detects when the batter in the bottle is beginning to run low and shuts down the system. This is important as if the system is run with no batter, the high-pressure air will spit out any residue in the tubes and uncontrollably spray batter (see Figure 13)³.



Figure 13: Splatter caused by a lack of batter

2.3 Environmental interaction and motor use

Though the use of active PID's Sputter can overcome simple environmental obstacles. This is done by using the coordinate system to recognize the location of Sputter and a PID to direct Sputter desired location. If something knocks the gantry or if there is debris on the rails sputter will keep moving until it reaches its destination. By using a direct PID system over the traditional 'motor spin for x time' approach,

³ This was an initial issue which we ran into while testing, which is also where the robot got its name from

the robot is much better at adapting to its surroundings, as inconsistencies in sliding resistance or impedances on movement won't result in location errors. This approach is also more accurate, as the robot is carrying a heavy gantry with lots of momentum, so a 'motor spin for x distance' would often result in overshooting the target. Using constant feedback helps us combat this issue by being able to correct the overshoot and move to the exact location desired.

All the motors on Sputter use custom 3d printed housings and connectors to have increased tolerance and decreased weight over the stock VEX parts. Another advantage of this approach is that the motors are held much more securely than the given VEX mounts. This allows us to have higher accuracy when moving the gantry while also being able to minimize the weight of the system so that the low-powered VEX motors can drive the system. Through the combination of tight, lightweight design and PID's, means the motors can be driven to a highly accurate degree, allowing it to carry out the complex tasks required.

The robust motor system that was developed works extremely consistently; however, on the edge cases where the system does not hold up, and environmental factors cause malfunctions, the team have the designated emergency stop to pull away from the hotplate and shut down all systems. This is the last layer of defense in case of failure and ensures that no matter what the environmental changes are, the robot will keep vital systems safe.

2.4 Recognition of completed tasks

Sputter uses its PID along with wait functions as its primary way of recognizing task completion. Originally, the robot was going to incorporate extra sensors to be able to check for the location of the plate and how cooked the pancake is; however, after some testing, it was determined that these additional systems were much more inconsistent than using the bounds of each axis. It was also found that a timer was more accurate in creating a consistent and well-cooked pancake than using the color sensor to check for browning. Thus, Sputter will move on to the next task either if it has reached the prescribed coordinates or if the wait timer for the specific phase is complete. Through this combination, Sputter can consistently complete and move on to the next task as needed.

2.5 Scope Changes

When first developing the idea for what the robot should do, the team was developing a solution-based approach rather than a problem-based approach. This led the original scope and project to be focused on creating a modular arm that could switch out attachments to be used for a wide range of tasks. This idea was interesting in thought; however, it did not solve any problem and so would not be a viable design. With the guidance of Griffin Wilson, the group changed its scope to focus on a more specific problem and

from there, would go about coming up with solutions. After the 'informal presentation', the group decided on the problem that many of us did not have enough time to make breakfast. After doing further research as described in 1.1 The Problem, the team developed an understanding of the importance of breakfast, which further guided us in confirming that this project would be a valuable problem to solve. From this more specific scope, the team was able to develop more detailed solutions to the problem, which ultimately led us to the goal of creating a robot that would autonomously make a pancake.

The broad scope of "Our project [being] designed to address a broad range of problems by replacing the need for humans in repetitive, tedious tasks." [5] was changed to 'not having enough time in the morning to make breakfast', which would be solved by making an automated pancake robot.

3.0 Constraints and Criteria

3.1 Constraints

The major constraints faced while building Sputter were:

- Limited boundaries (Inside box)
- Motor strength
- Vex controls, such as sensors, are not very accurate.
- Limited number of motors and cables

The major constraint behind any cartesian robot is that its workspace must be within the bounds of the frame. This limits the maximum size of the pancake, resulting in the system needing to be much more precise to work in the small volume. The largest problem this causes is having enough room to get behind the pancake. The team calculated the required dimensions as follows:

Let w = pancake diameter, x, y, z be dimensions of range as depth, width, and height respectively

with the spatula being oriented in the x direction

$$x = 3w + 0.2w$$

$$y = 2w + 0.2w$$

$$z = 1.5w + 0.2w$$

Thus, if the desired pancake is 3in, the robot would need a work volume of 9.6 x 6.6 x 5.1 in.

This formula was derived as the robot would need twice the width of the pancake in the depth and height directions to be able to get behind, under, and lift the pancake to be flipped, plus an additional depth of one diameter to reach over and plate the pancake. The width is calculated by taking one and a half times the diameter of the pancake, as when flipped, the pancake shifts over to the left by half a diameter. The addition of $0.2w$ is to act as a buffer to allow for some error in dispensing and flipping.

VEX IQ motors only have a maximum output of 1.4W continuous and a maximum torque output of 0.414 Nm. This means that the y-axis motor with a 1cm large rack and pinion system can carry the following weight:

$$Force = \frac{0.414Nm}{0.005m} = 82.8 N$$
$$Mass = \frac{82.8N}{9.81 \frac{m}{s^2}} = 8.44 kg$$

With a reasonable safety factor of 1.5, along with the added resistance caused by the sliding side, the y and z axes, and the entire dispenser and flipper systems, can only weigh approximately 4.5kg. This means that when developing the robot, the team must try to keep the weight down in as many areas as possible.

VEX sensors are inaccurate, which constrains the maximum precision that can be achieved from the robot and can significantly impact the consistency of the pancakes. To combat this constraint as best as possible, the robot uses PIDs and active feedback wherever possible to create self-correcting functions that should accommodate the inaccuracies. Along with improving the code, the team also aimed at making the physical design as rigid and accurate as possible. This was achieved by primarily using custom 3d printed parts over the stock VEX pieces, and by implementing reinforcing aluminum solid rounds when needed.

The most challenging constraint for the robot was that the team had very limited access to motors and cables. The robot could only be made of four motors, three of which had to be designated to the three axes of the cartesian system, meaning only one was available to be used on the dispenser and flipper system. Another constraint that was added while building and testing is that by only having one 600mm cable, the robot was significantly restricted in the range of motion, especially for any sensors or motors that needed to be placed on the flipper carriage.

Each of these constraints helped guide the design of the robot; however, the motor and wire quantity was the most critical in decision-making. Due to only having one motor for both functions of flipping and dispensing, initially the team developed a Geneva Mechanism⁴ and later an extreme gear-down system to allow the robot to do each task independently. Furthermore, the lack of 600mm wires meant that the robot could only have one motor or sensor that could access the entire volume, which is why the height is only measured using the coordinates of the z-axis instead of having a reinforcing distance sensor. The cable length was the primary factor in the work volume, which made the pancake size constraint not valuable,

⁴ A Geneva Mechanism allows a rotation by a circular disc which once per rotation drives another gear by precisely 1/3 of a rotation

as the size of the pancake would be determined by the length of the cable rather than the pancake size determining the work volume.

3.2 Criteria

The two criteria for the success of Sputter were that it could minimize human intervention while maximizing consistency. Both criteria were important guiding factors behind the design of Sputter and were a large reason for the extremely complex dispensing system that allowed us to have a fully automated food process. These criteria remained consistent throughout the development of Sputter, with no additional criteria added or changed. By minimizing human intervention, the robot was optimized to solve the problem it was built for, as the final product required no interaction except for starting the robot up. Furthermore, by optimizing consistency, Sputter was able to fully make and plate a pancake at a consistency of 92% and even when failure occurred, the result was often still edible but had some defects. Through a combination of both these criteria, the robot was able to be specifically designed for the problem and optimized the robot to fit the solution required.

4.0 Mechanical Design and Implementation

Sputter was developed through a modular system design, which helped us design, build, and test the robot more effectively. This section will explore how this approach improved the robot, along with looking at how Sputter and each of its mechanisms work. This section will also go over the design choices and iterations made along the way, with analysis and justification behind the choices. The section will also investigate the proactive techniques, including using Computer Aided Design to model out the whole system before manufacturing and using modularity as much as possible, so that the robot would need less manufacturing time while also being extremely easy to iterate.

4.1 Overall Design

Sputter is, at its core, a 3-axis Cartesian robot, like what is seen on many 3d printers or CNC machines. The core movement is handled by this cartesian system and is what allows the gantry to move around the entire workspace in all three dimensions (see Figure 14). The y-axis works by a rack and pinion on one side with a slider on the other. The X and Z axes are near-identical copies of each other, both working by using a central rack and pinion that slides on two 3/16in aluminum solid round stock.

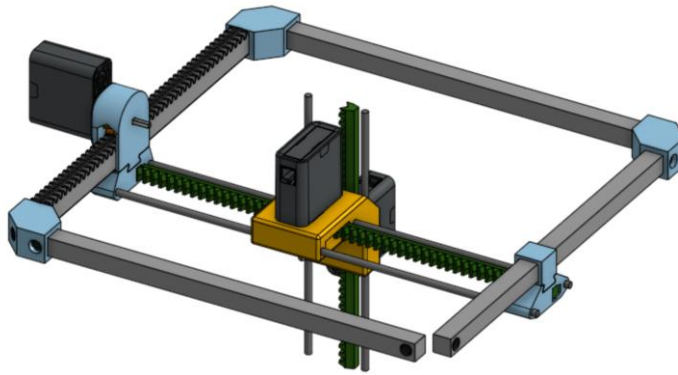


Figure 14: Frame Assembly

Most of the complex work to complete the pancake-making is done by the extrusion head and flipper. This system is modular and interchangeable by connecting to a cage that houses the mechanisms, which get pinned to the cartesian (see Figure 15). Due to the limitations of only one motor for this whole system, the robot works by using a 1:1 gearing for the flipper and a 5.33:1 ratio for the valve so that a small 180-degree rotation of the spatula wouldn't open the valve, but multiple rotations would cause flow.

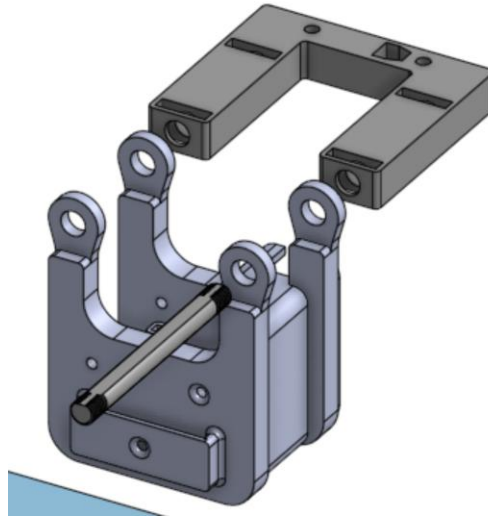


Figure 15: Extrusion and Flipper carrying cage

4.2 Subsection Cartesian System

Frame

The printer's frame underwent two major design iterations:

4.2.1 TETRIX Beams Approach

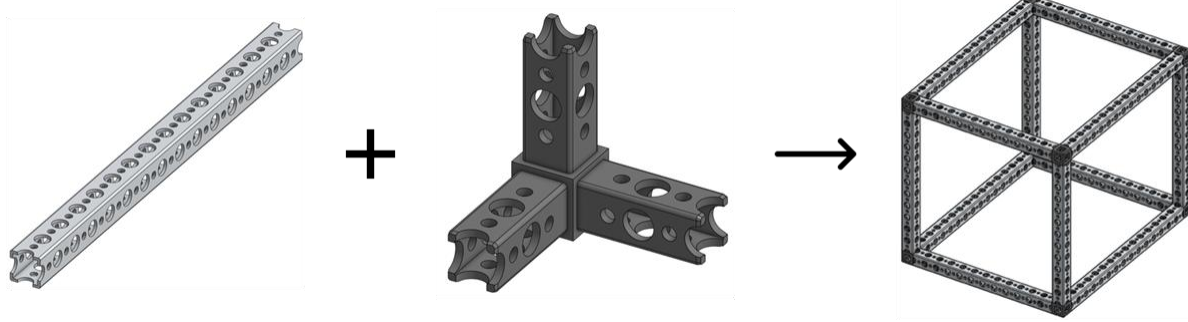


Figure 16: TETRIX Frame Assembly

The team initially believed that using TETRIX aluminum beams and corner connectors from the parts kit would provide a fast and reliable way to build the printer frame. A parametric configuration model of the frame was thus created using the number of holes per beam as input variables for all three dimensions (X Y Z).

However, this approach introduced many issues and constraints in the design.

First, using these beams imposed a 14-mm-fixed-width constraint on the sliding mechanism, impeding experimentation on what size and shape would be optimal.

Second, modularity was difficult, since the beam have a fixed length of 16 mm between each hole and would only come in certain configurations (4, 5, 6, 7, 8, 13, or 15 holes).

Third, the assembly was very unstable, as connecting the beams with the parts available in the TETRIX kit introduced wobble in the joints due to major tolerance differences. To solve this problem, printed M8 screws (see Figure 17) were used to design custom brackets to better hold the frame and reduce wobble. These brackets each contained two attachment points on each side to prevent the attachment points to act as pivots.



Figure 17: TETRIX Frame Holder

4.2.2 Fully Customized Approach

After experiencing the inconveniences of using the TETRIX parts and realizing that the bottom portion of the frame was not needed – and thus, neither were the vertical beams – the team determined that the TETRIX aluminum beams were not needed and that a fully customized, 3D-printed approach would increase design freedom and remove several constraints, including fixed beam width, length, and parts attachment points (see Figure 18).

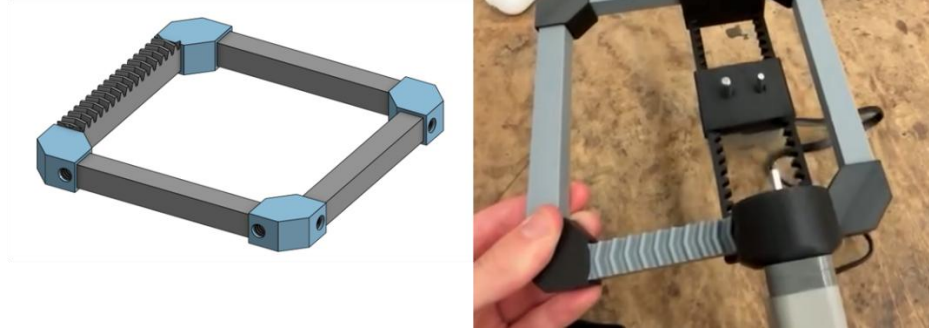


Figure 18: Printed Frame

Figure 18 shows a test small test print to validate and verify that the mechanism functions as intended. This modification resulted in a very stable, rigid, modular and customizable frame, which was entirely printed using PLA (Polylactic Acid) filament.

Another advantage of this design is that the rack used in the frame crawler mechanism now became a structural component of the frame.

Since PLA is prone to creep, threaded holes were added in the beams to provide a backup fastening method in case the interference fit loosens over time.

4.2.3 Linear Motion Mechanism Designs

Mechanism Decision Analysis

Several suitable design options for the 3-axis linear motion mechanisms were analyzed and discussed in the research phase of the design cycle. The main options were the following:

	Rack and pinion	Lead screw	Core XY
Pros	• Simple to design • Only 2 components • Easy to print and assemble • Customizable and modular	• High positional accuracy • High torque • Self-locking	• Fast • Stable • Precise • Industry standard • Reduced gantry mass
Cons	• Medium speed • Risk of backlash • Heavy moving assembly	• Slow linear speed • Lots of friction on printed parts • Heavier	• Complexifies code • Requires timing belts • Complex belt routing • Under constant tension load (could break)

For the sake of fast iterations, ease of design, modularity and design exploration, the team decided to use a rack and pinion system to create a crawler mechanism on all three axes. The Core XY option was put aside due to the added complexity of belt tensioning and routing, and the lead screw option would not have

offered a sufficiently high speed. That said, in retrospect, replacing the cartesian system with a Core XY mechanism would have made the printer more reliable and would have offered an industry-relevant learning opportunity.

All racks and pinions used a herringbone design; a convenient design improvement enabled by additive manufacturing. Below is a list of the relevant advantages of this design:

- Rack and gears stay perfectly aligned (self-centering)
- Quieter, smoother operation
- More torque
- Reduced backlash

4.2.4 Y-Axis Mechanism / Frame Crawler

The Y-Axis mechanism (a.k.a. **frame crawler**) defines the movement along the frame. It was the first mechanism to be printed and tested since the rack was also used as a structural component of the frame. It was comprised of two sections: the frame crawler and the slider.

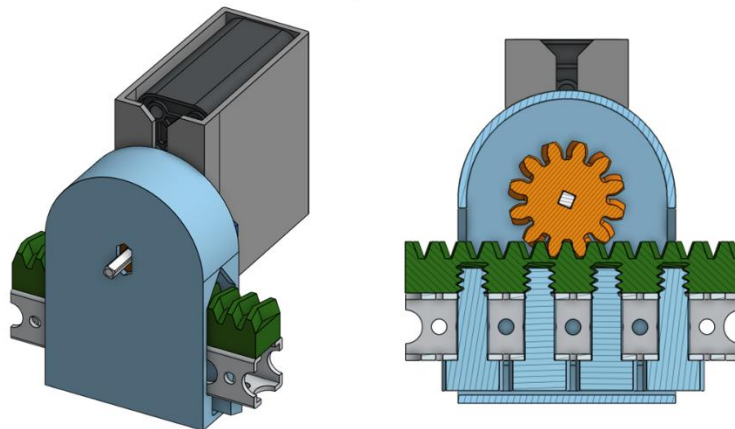


Figure 19: Frame Crawler Mechanism

In the first design, when the frame was still made using the TETRIS beams, the rack was mounted on the frame using the printed screws shown in Figure 17 and threads embedded directly into the thickness of the part (see Figure 19).

This initial version also contained a motor enclosure to hold the motor in place. It made the crawler bulkier since the enclosure needed to be mounted onto the slider using screws. Because of the limited available space, threads could not be embedded directly in the part to mount the motor without making the overall crawler significantly larger. Keeping the crawler as small as possible was necessary to conserve a relatively small moment of the parts and maximize the available space for the crawler's motion.

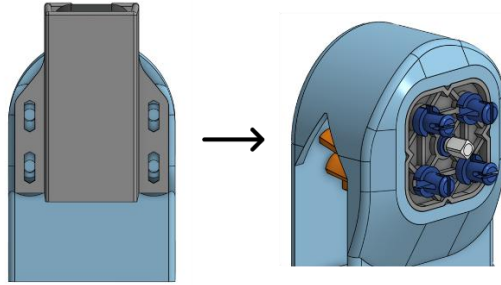


Figure 20: Motor Attachment Methods

Upon determining a proper way to attach the motor shaft to the VEX motor using the provided part kit, the decision to eliminate the motor enclosure was made. The attachment point between the crawler and the motor was thus replaced by an interference fit of the VEX square bracket directly into the crawler part. This radically simplified both design and assembly.

The crawler mechanism was responsible for the motion along the Y-axis, but a sliding guide was needed on the opposite parallel beam so that the transversal crawler mechanism would be parallel to the X beams. A simple square slider (see Figure 21) was used, as the friction between 3D printed PLA is minimal, especially when the slider motion is parallel to the layer lines.

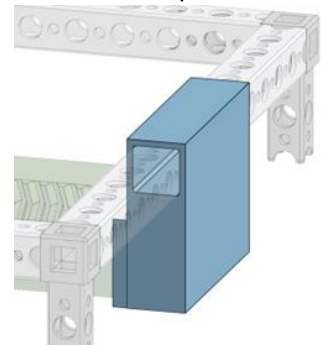


Figure 21: Frame Slider

4.2.5 X-Axis Mechanism / Transversal Crawler

While still using a rack and pinion, the mechanism for the X-axis (a.k.a. **transversal crawler**) was designed slightly differently, since the transversal assembly needed to be quite rigid and stable.

To achieve that rigidity, a double rack and pinion system was used. The transversal crawler was comprised of two gears – a drive pinion and a driven pinion – attached to the slider using metal shafts. To attach the motor, the same interference fit as observed in Figure 20.

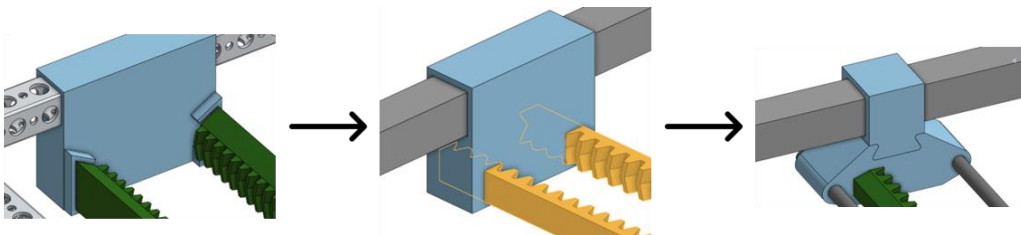


Figure 22: Evolution of the Connection between Transversal Rack and Frame Slider

Several designs were explored to attach the transversal crawler to the frame crawler. The first one was an interference fit of the trapezoidal racks in slots created on the slider. The second was an interference fit of the racks directly embedded into the part. After testing the system, it was observed that the racks were not rigid enough to act as a structural component of the transversal crawler assembly. Thus, in the 3rd version, aluminum rods were implemented to ensure the assembly would stay straight, and the dual rack and pinion

system was replaced by a single rack now solely responsible for the motion of the assembly (see the evolution in Figure 22).

As can be seen in the 3rd image of Figure 22, the final frame slider was made significantly smaller than the previous ones. This is because, even with the increased rigidity provided by the aluminum rods, the assembly still had trouble moving when reversing direction because a torque was created between the slider and the frame beam, causing a jittery motion. Reducing the length of the slider decreased the moment arm, and by extension decreased the jittery motion, which made the operation of the system much smoother.

Z-Axis Mechanism

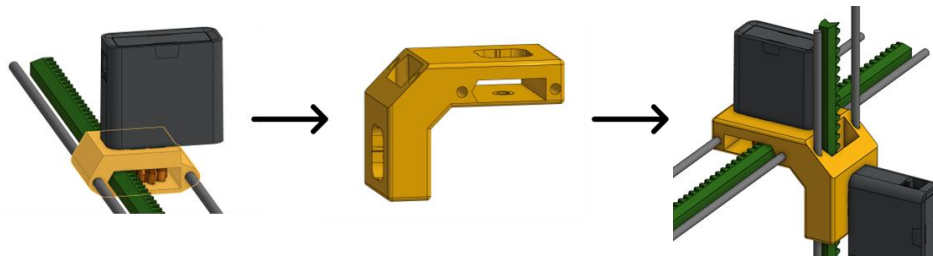


Figure 23: Z-Axis Bracket Design. Left picture shows the initial single-axis transversal crawler system when the Z-axis mechanism was not implemented yet. Center picture shows how the two instances of that left part were connected. Right picture shows the final assembly of the X-axis and the Z-axis mechanisms

The Z-axis mechanism was an exact copy-paste of the transversal crawler mechanism. The original transversal slider bracket was duplicated, set vertically, and the two instances were then connected together (see Figure 23).

4.3 Subsection Nozzle and Flipper

In designing the nozzle that was required to extrude batter and flip a pancake, the most important constraint was that there was only one motor available to actuate both functions of the part.

To solve this problem, the first version of the nozzle relied on a three-slot Geneva Mechanism. The axle of the Geneva mesh had a partly toothed gear that spun with the mesh, and only transmitted power when it was at a certain position in the Geneva Mechanism (see Figure 24). This positional control enabled a ball valve to be in an on or off state while still being able to freely spin the axle connected to the motor, so long as it rotated less than two full rotations. This way, a pancake could be flipped without extruding batter, meeting the primary design constraint of achieving both functions of the nozzle with one motor. After testing the part, it was found that the flipping mechanism could not produce the requisite

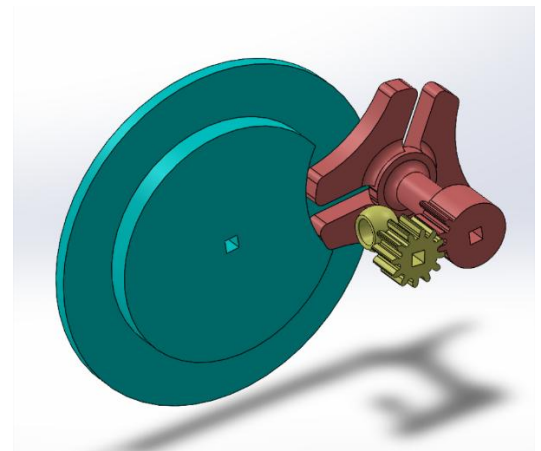


Figure 24: Geneva Mechanism and Ball Valve

torque to flip a pancake. This came as the result of a combination of two factors: the length of the moment arm of the flipper mechanism and the fact that VEX motor stall torque is proportional to motor speed.

In the second design iteration, the problems with the flipper mechanism were solved by gearing down the motor and moving the axle that spun with the spatula closer to the bottom of the nozzle. By gearing down the motor, it could spin faster and with more torque while keeping the spatula's angular speed the same as in the previous version. This helped the spatula to be able to fully rotate with a pancake. Next, as can be seen in Figure 25, the axle closer to the bottom of the spatula allowed for the effective inertial arm of the nozzle to be reduced considerably. This allowed the spatula to spin with greater force and flip heavier pancakes. During testing, the flipping mechanism could flip pancakes consistently and reliably; however, upon testing the extrusion mechanism, it was found that there were several leaks in the 3D printed ball valve.

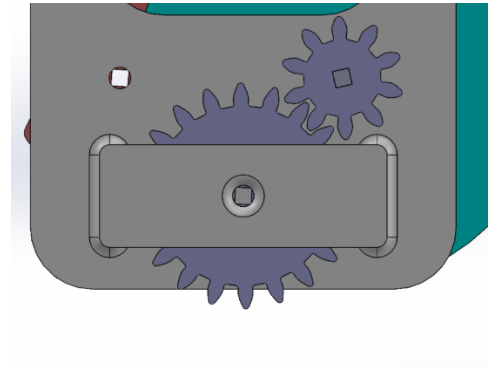


Figure 25: Motor Transmission to Geared Down Axle

In the third design iteration, the 3D printed ball valve was redesigned to be fastened by two M4 screws and one aligning slot, compared to the previous version, which used two aligning slots and one M4 screw (see Figure 26). The additional screw helped to press the top of the ball valve enclosure against the bottom of the enclosure, providing a better seal, and making it tight to pancake batter being passively poured through the valve. However, upon full system testing under pressure, the pancake batter was pushed through parts of the ball valve that were not completely airtight. When tested without pressure, the adhesive and cohesive forces between the pancake batter and the surrounding PVC tube could not be overcome, and the batter could not be pushed to the end of the nozzle. To solve this problem, the plastic pieces surrounding the ball valve were fused with a soldering iron. This stopped leakage from this part of the valve, but the area between the surrounding ball valve enclosure and the ball valve still leaked (see Figure 26). To seal this part of the valve, hot glue was applied to function similarly to an O-ring. Despite this, pancake batter leaks were still present under pressure.

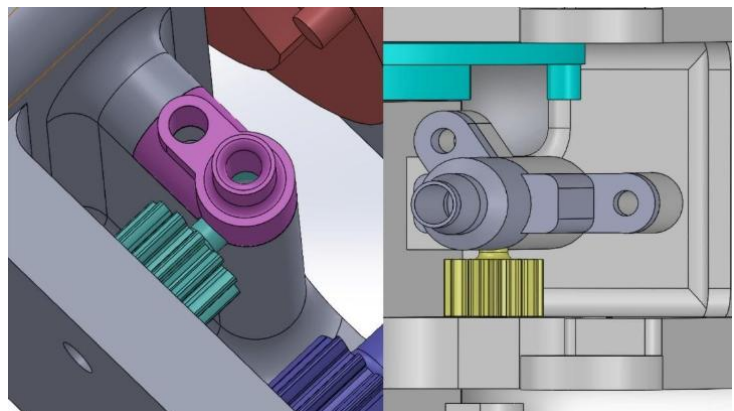


Figure 26: Imaged on Left is Ball Valve Fastening of Version 2 with one M4 Hole, Imaged on Right is Ball Valve Fastening of Version 3 with two M4 Holes

In the fourth iteration of the part, a brass ball valve was used instead of a 3D printed ball valve (see Figure 27). The motor had its power transmitted by coupling a shaft to the hex nut on the ball valve. Additionally, instead of relying upon a Geneva Mechanism, as in previous designs, the power transmitted to the ball valve was reduced a great deal by a gear train, such that the ball valve would turn very marginally for a full flip of the spatula. This way, a pancake could be flipped without opening the brass ball valve enough to allow batter to pass through it and could be opened by effectively flipping the spatula many times. Upon testing, the brass valve was able to prevent the passage of batter under pressure and without leaks and allow batter to extrude when opened.

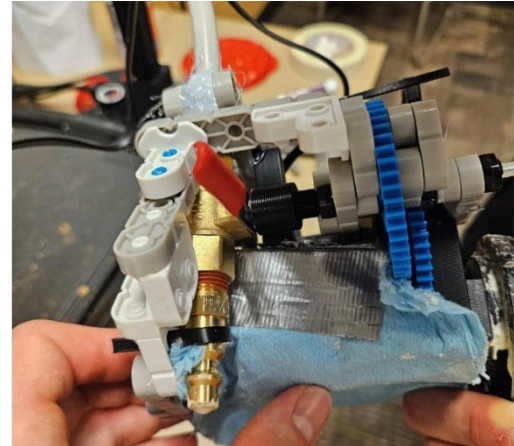


Figure 27: Brass Ball Valve for Extrusion of Pancake Batter

4.4 Subsection Pneumatic Pressure System

Pancake batter has an extremely viscous liquid, so it does not flow through tubes under gravity. For the extrusion system to work, there needs to be a relatively high-power feed system that can supply a steady flow of batter to the extrusion nozzle shown in Figure 27. Furthermore, the supply tube is a 1/4in PVC flexible tube, which further increases the resistance the batter faces when reaching the extrusion head. To solve this issue, the team designed a custom cap that could be attached to a standard 2L bottle. This cap has two connections on it; one is an M5 threaded air quick connector, which supplies pressurized air into the bottle, and the other is a 1/4in barbed connector, which the tube batter supply gets press fit and glued onto (see Figure 28). With the use of this custom cap, the robot can supply pressurized air to act as a pushing force that drives the batter out of the tube and to the extrusion head at a reasonable flow rate. To ensure airtightness, this cap was also built with a seal ring into which an O-ring could be press fit⁵.

Finally, a continuous supply of constant-pressure air is needed to power the system. To achieve this, the robot has a pressure hub (see Figure 3) that has a metal tank, which gets pumped up to a maximum of 100psi. This tank then goes through a regulator to step the pressure down to roughly 20psi. After the regulator, the air runs through a manual valve to ensure safety and allow for bleeding of the system when done with use. The manual valve was then connected to the small M5 quick connector on the side of the two-litre bottle.



Figure 28: Custom Bottle Connector

Blue arrow is the M5 hole for the quick connector

Red arrow is the 1/4 in barbed connector

⁵ The core O-ring part was made using public files from www.uswaterrockets.com, and the group designed all parts of Figure 28 in black for their needs

5. Software Design and Implementation

This section describes the design of the software through a description of the functions used, how the software was tested, and any tradeoffs of design choices made for the Pancake Making Robot. The main goal of the software was to maximize consistency of motor movements, allow for simultaneous movement of all four motors, and allow for simple, intuitive programming of the autonomous pancake cooking process.

5.1. Overall Software Design and Architecture

The program was written in C++ inside the VEXcode IQ IDE and is designed so all movements can occur simultaneously. The program is structured around a real-time loop inside the `waitFor()` function which controls all of the motors, and updates all of the sensors. Motion is handled by setting target positions for each of the motors inside the main function, and the `waitFor()` function continuously updates motor speeds using a P controller until the desired time has elapsed.

The ability to control the 3D position of the robot is significant since this allows the robot to follow G-code commands and print shapes generated in a 2D drawing software, like AutoCAD. To accomplish this, the x,y, and z motor encoder values are converted into millimetres using predefined constants (XDEGTOMM, YDEGTOMM, ZDEGTOMM) which mean commands can be given in physical units, comparable to G-code.

5.1.1. Program Blocks

The software for the Pancake Making Robot was separated into the following blocks:

5.1.1.1. Homing/Zeroing

These functions move the motors in reverse until they collide with the ends of the linear rails. This collision is detected when the motor speed drops below 10% and the motor amperage is above a tuned threshold, 0.5 A. When the collision is detected, the motor resets the internal encoders to zero.

These functions were implemented to ensure the robot has a reliable and repeatable starting point. This makes sure that position movements are precise and replicable no matter the starting configuration of the robot.

5.1.1.2. PID Control Loop

The `waitFor(int millisec)` function calculates the motor error and a proportional motor speed for the x, y, z, and flipper motors on a consistent timescale, 10 ms.

This function was implemented to allow for simultaneous and smooth movement for all motors. The P controller increases accuracy and decreases oscillations compared to a more typical bang bang controller.

5.1.1.3. Emergency Handling

The `checkEmergency(bumper& B)` checks if the emergency stop button has been pressed. It contains a simple debouncer to eliminate accidental triggers from sensor errors.

This check was implemented to provide a safety shutdown procedure. If the bumper sensor is pressed at any point in operation, the robot immediately stops all motors, removes the z-axis from the hot plate, and plays an alarm to alert the user.

5.1.1.4. Wait & Display

The `WaitCook(int sec)` function draws a dynamic progress bar as well as a count down timer onto the Brain screen.

This function was implemented to create a user-friendly system that informs the user of the time remaining until the pancake finishes cooking. This function calls the `waitFor` function to ensure that all sensors continue to update.

5.1.2. Task List For The Pancake Preparation Demo

The task list for the demo evolved throughout the build process as functionality was evaluated.

5.1.2.1. Finalized Task List

The following sequence of tasks is executed autonomously once the program has started:

1. Wait for TouchLED to be pressed.
2. Home gantry motors.
3. Move to the pancake location and dispense batter.
4. Move the Z axis away from the hotplate and wait for the pancake to cook while displaying the countdown timer and progress bar.
5. Move to the pancake location and flip the pancake.
6. Move the Z axis away from the hotplate and wait for the second side of the pancake to cook while displaying the countdown timer and progress bar.
7. Move to the pancake location, pick up the pancake, and flip the pancake onto the plate.
8. Home gantry motors and wait for the next TouchLED input. Close the program if the distance sensor is pressed.

5.1.2.2 Task List Changes and Rationale

Originally, the task list stated that the robot would cook 5 pancakes before shutting down automatically. This was changed to cook pancakes until the user presses the distance sensor to end the program, since the user may want to eat more or fewer than 5 pancakes.

5.1.3 Data Storage Within the Program

The program stores tuned values, like KP, in constant variables, which ensure that they are not changed during the program runtime. All information pertaining to the motors and sensors are contained within the respective classes and retrieved when needed for calculations and checks. The x, y, and z targets are stored in mm to allow for G-code integration.

5.2. Function Documentation

All complex control logic is encapsulated in the following core functions:

5.2.1 `waitFor`

Parameters: the number of milliseconds to wait for, as an integer.

Return type: void.

How it works: See Figure 29

Author: Mathew

.

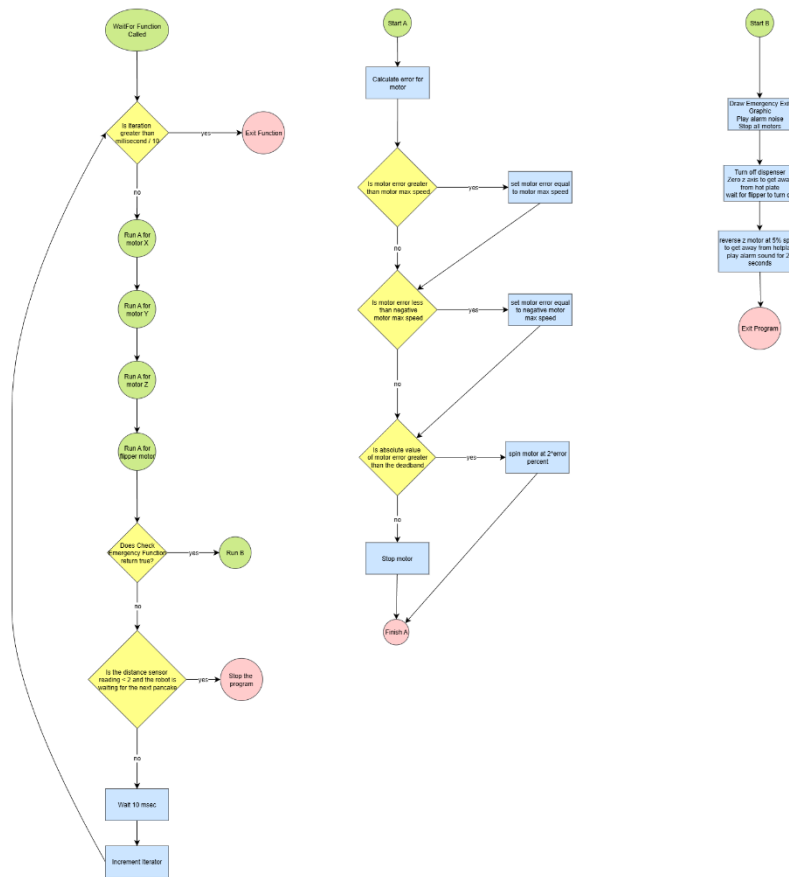


Figure 29: Wait for

5.2.2 WaitSteady

Parameters: no parameters.

Return type: void.

How it works: See Figure 30

Author: Felix.

5.2.3 checkEmergency

Parameters: the reference of a bumper sensor.

Return type: boolean.

How it works: See Figure 31

Author: Aydin.

5.2.4 Zero

Parameters: the reference of a motor.

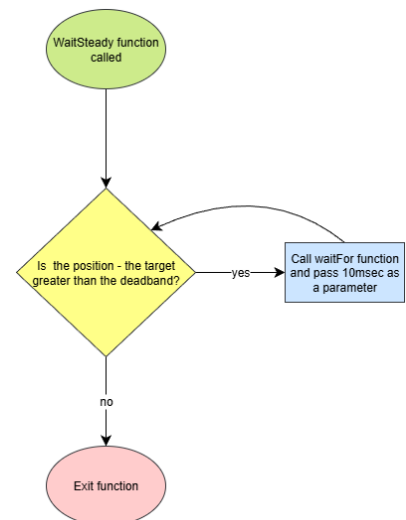


Figure 30: Wait Steady

Return type: void.

How it works: See Figure 35

Author: Lennart.

5.2.5 *ZeroAll*

Parameters: none.

Return type: void.

How it works: See Figure 34

Author: Matthew.

5.2.6 *WaitCook*

Parameters: the number of seconds to wait for as an integer.

Return type: void.

How it works: See Figure 33

Author: Mathew

4.2.7 *dispenseBatter*

Parameters: the number of seconds to dispense for as an integer.

Return type: boolean.

How it works: See Figure 32

Author: Lennart.

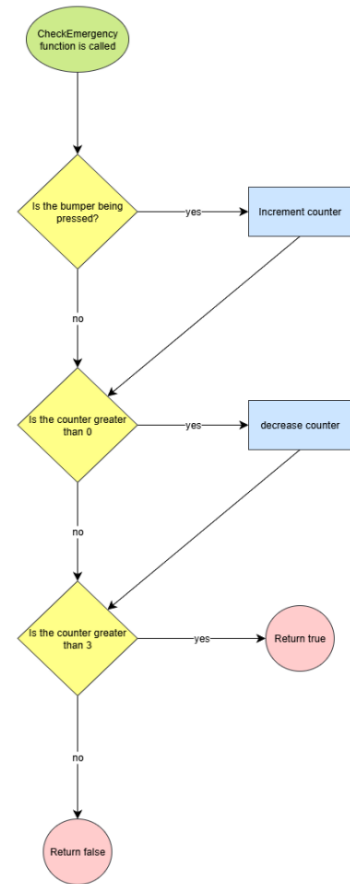


Figure 31: Check Emergency

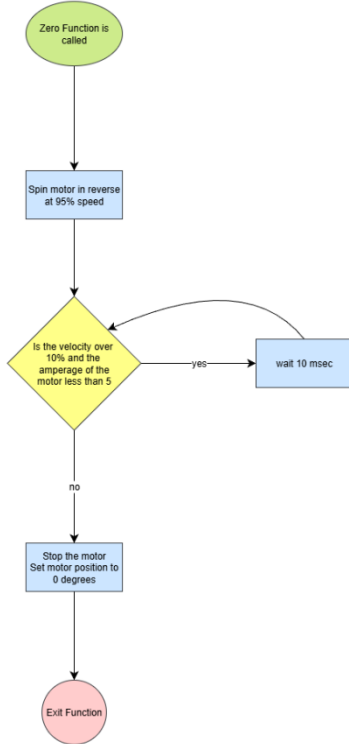


Figure 35: Zero Function

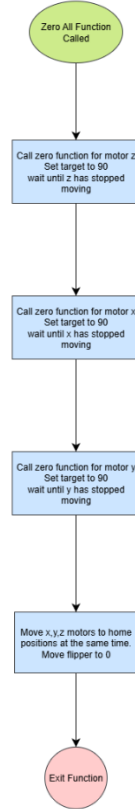


Figure 34: Zero All functions

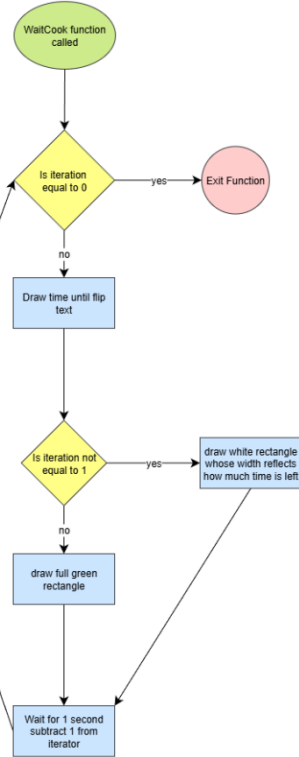


Figure 33: Wait Cook

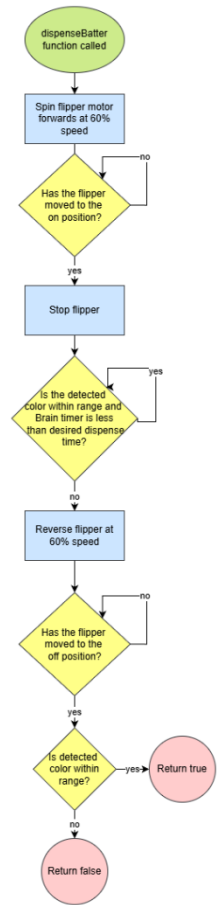


Figure 32: Dispense Batter

5.3. Software Design Decisions and Trade-offs

Decision	Explanation	Trade-off
Motor Position Converted to Millimetres	This allows for the implementation of G-code commands, and simple autonomous programming.	Requires careful tuning and measurement of conversion constants. The conversion constants are dependent on the physical system. Any change in gearing would require recalculating conversion constants.
Use of Stall Detection for Homing	Since the gantry motors could be started in any position, the robot needs to home the motor positions. The only available sensors were inside the motor as there were not	Slight inaccuracies in real world position due to the flexibility of the 3D printed hardstop and the vex axes.

	enough bumper sensors to create a limit switch style homing system.	
Proportional Only Motor Control	The motors only move at a speed proportional to their error. This allows for very straightforward PID exit conditions. This also simplifies calculations within the waitFor loop and results in less tuning needed for consistent motion.	Less control over gantry motion profiles. No ability to recover from steady-state errors.
User interface on the Brain Screen	A relatively high detail user interface conveys information to the user that would be impractical with the TouchLED. Allows the user to see what state the robot is in, and how long left when in the cooking phase.	Extremely small screen size makes information hard to see.

Table 1: Software Design Decisions

5.4. Software Testing Procedure

Software testing took place iteratively, ensuring that basic functions are reliable before integrating into a final cohesive autonomous cooking sequence.

5.4.1 Individual Function Testing

Function	Test Case	Expected Behavior	Validation
Zero()	Motor is positioned away from the homing edge	Motor moves until it collides with the edge, then stops and resets the encoder position	Motor position was printed on the Brain screen and showed a value near zero when it stopped
ZeroAll()	Gantry begins in an arbitrary position	Z-axis is homed first, then moved to a safe location. X and y axis are homed after. All axes move to a predefined home location	Observed that each motor completed the homing motion and moved to the appropriate location
waitSteady()	A target position is set for the motor, then the function is called	Code doesn't execute the next line until the motor has completed its movement	The motor moved to different positions one after the other, starting its next movement when it completed the last
waitFor()	Function is called during the WaitSteady function testing	Updates motor speeds and sensor inputs. Emergency stop should activate when bumper sensor is pressed	Motor changes speed proportional to error, and emergency stop was activated when bumper sensor was pressed
checkEmergency()	Function is called when WaitFor is called	Returns a true value when bumper is pressed for 3 consecutive ticks	Emergency stop graphic and sounds played when bumper sensor was pressed

DispenseOn()	Flipper is initially in the closed position	Flipper spins to the calibrated angle to open the extrusion valve	Verified that the flipper rotated to the appropriate angle
WaitCook()	Function is called at the beginning of main with a wait time of 10 seconds	Screen displays a countdown from 10 as well as a dynamic progress bar	Confirmed that the progress bar and countdown timer updated every second, and waited for the full 10 seconds

Table 2: Individual Function Testing

5.4.2 Integration Testing

The final phase of testing integrated all of the functions into the main function. The testing involved running the main() function to completion calling the functions in the following order. 1) ZeroAll() 2) waitFor() 3) checkEmergency() 4) DispenseOn() 5) waitSteady() 6) WaitCook(). The test is successful if the homing sequence is completed without error, the bumper sensor check returns the appropriate value, and the waitCook() function is called after the pancake batter extrusion valve is fully opened.

5.5. Significant Problems Encountered and Resolution

During testing, the Zero function occasionally failed to detect the end of the gantry reliably, leading to an unending loop in which it continuously moved into the end stop. This meant the next line of code would never be executed. The cause of this problem was the way the motor receives power. From observation, when the motor is stalled, it gradually ramps up power, but resets to zero when it detects it is not moving. This caused the amperage to rise, then fall, never surpassing the threshold. The voltage of the battery also had an impact on the consistency of motor homing. The solution to this problem was to lower the current threshold to 0.5 amps and to ensure the battery always has over 30% battery.

6.0 Verification

Sputter excelled at meeting every constraint and criterion for success. On top of meeting the initial constraints laid out at the beginning of the project, Sputter was also able to fulfill the additional constraints later added. Each constraint and criteria were met as follows:

6.1 Limited boundaries

Sputter was able to successfully perform all its necessary actions within the limited boundary of the cartesian system that was designed. A major reason for shifting away from the original TETRIX design, as stated above, was to better meet this constraint, as a fully customized and 3d printed frame allowed us to increase the work volume and fully complete the automation mode. However, future iterations of Sputter would involve increasing this build volume in the x y plane to allow for larger and multiple pancakes.

6.2 Motor Strength

After calculations and testing, it was found that by adjusting the gear ratio of the rack and pinion mechanisms, the robot would be able to move the heavy gantry while still going at a reasonable speed. By developing the robot as a modular design, the team was able to test what sizes would be needed to run the systems at maximum speed and reliability.

6.3 Vex control inaccuracies

Through testing, it was found that most of the vex sensors were not as inaccurate as previously assumed them to be. Furthermore, by optimizing code and tolerances (especially through the use of PIDs), the robot was able to have much higher precision when moving the gantry than expected. This level of accuracy also allowed us to draw complex shapes, such as stars, with reasonably high tolerances.

6.4 Limited number of motors and cables

This constraint ended up being the largest challenge when developing Sputter. The build volume and motor/sensors at the extrusion and flipper head were all decided by the fact that the robot could only have one 600mm cable that could reach the entire desired build volume. This was a large contributing factor to why the robot didn't have a secondary distance sensor to work to detect the distance of the hotplate; however, the team did manage to get around this problem by improving the coordinate system to get a reliable location. The other way the team managed this constraint is that the brain is hung by a long connector off the bottle cage (see Figure 6). Hanging the brain allows it to swing as the gantry moves, which extends the range of motion of the cables. On top of the short cables, the other difficult constraint was that the group only had access to four motors, three of which were needed for the cartesian. The solution for this constraint was to have both the flipping and dispensing functions operated by one complex motor system, as described in 4.3 Subsection Nozzle and Flipper.

6.5 Maximize consistency

Sputter's was 92% consistent with making pancakes start to finish. This is well above the goal that the team were aiming for and was achieved by maximizing both the physical and software aspects of the robot for consistency. By using a fully custom-printed frame and cartesian system, the robot could maximize the tolerances and minimize the backlash caused by stock parts. Then by implementing PIDs and an accurate coordinate system, the robot could have precise movement that allowed Sputter to execute on such a complex task very consistently.

6.6 Minimize human interaction

Sputter could successfully dispense, cook, flip, and plate a pancake with no interaction needed other than filling up and starting the robot. This is a complete success in fulfilling the criteria, as a user can start the robot as soon as they wake up and will not need to interact with it until they want to eat. Some features that were developed but were not implemented into this prototype are the ability to make multiple pancakes and set a time to start making them. These two implementations would allow for full autonomy of breakfast every night; however were not practical for the prototype as they did not demonstrate any extra functionality and could potentially be dangerous.

7.0 Project Plan

7.1 Task Assignments

The team divided work by subsystem. Felix designed and built the three-axis gantry. Lennart was in charge of the robot support structure, the electronics mounting, and the batter extrusion and storage system. Aydin designed the pancake-flipping mechanism and made the active end of the extrusion system. Matthew handled all software design, testing, and implementation. This specialization and separation of the design allowed each subsystem to be developed efficiently before final integration.

7.2 Timeline Overview

A Gantt chart was created at the start of the project to lay out key deadlines and major milestones. Many of the tasks stayed on schedule, including the programming of the basic control classes, advanced motor control, 3D printing and prototyping, autonomous specific tasks, and the final build and testing. The team completed everything on time for Demo Day.

Some design phases extended beyond the original plan. CAD work and designs for the batter storage system continued into mid-November, mainly due to major hardware redesigns throughout October as the team adjusted several plans to expand the scope of their pancake-making robot.

7.3 Plan Revisions and Team Coordination

The gantry and batter extrusion systems were redesigned multiple times, with the final design selected after the formal presentation in the first week of November. These revisions did not impact the final deadline. This subsection approach was also valuable in speed of testing, the entire robot would not have to be rebuilt after every change, and allowed us to rapidly prototype. Team coordination was focused on allowing each member to work independently on their subsystem and then integrate near the end, ensuring straightforward prototyping and testing.

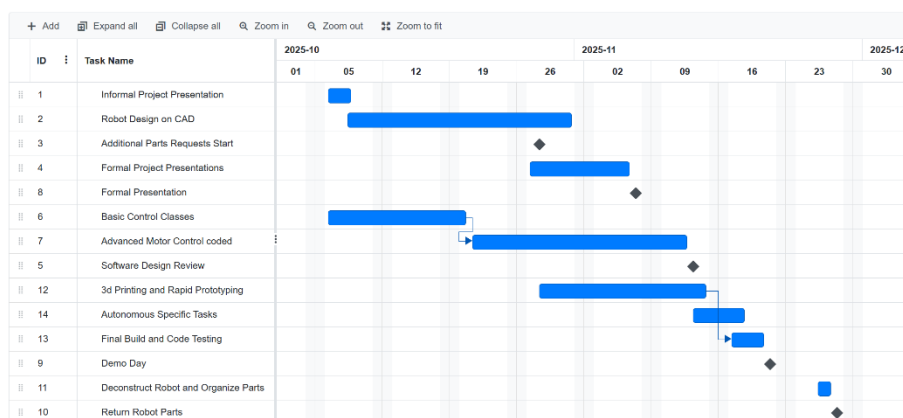


Figure 36: Original Timeline

8.0 Conclusion

The project purpose was to build a robot that could dispense, flip and plate a complete pancake with minimal user involvement, while remaining within the limitations of the VEX IQ system. By the end of the project, a robot was produced that could reliably dispense batter, flip the pancake after a tuned duration for adequate cooking, and plate the pancake without significant manual intervention. This performance met the initially established success criteria and demonstrated that the mechanical and software systems of the robot were able to work as intended.

Mechanically, Sputter went through several design cycles before reaching its final form. The supporting frame, gantry, nozzle, and pressure system were all iterated upon as testing revealed new problems and limitations. The early switch from TETRIX components to an adjustable and fully 3D printed frame allowed for higher rigidity and greater modularity for testing. The nozzle and flipper system saw several changes, moving from relying on a Geneva Mechanism that leaked at many points in its 3D printed ball valve to a reducing gearbox with a brass valve. These iterations enabled the system to flip pancakes consistently and reliably turn batter dispensing on and off.

The software was designed around the idea of keeping motion smooth and precise using PID control. Functions were designed for homing, emergency stopping, movement control, batter dispensing, and cooking time. Testing was first conducted with each function individually, before integrating all aspects of the software design together. Early issues involving zeroing the robot reliably with the detection of motor stalling were solved by adjusting current thresholds and battery requirements.

To conclude, the finished system successfully demonstrated that an autonomous pancake robot is achievable using only four VEX motors, limited sensor options, and a constrained workspace. Through

mechanical redesigns, tuning and calibration, and specific software development, a robot was produced that operated safely, consistently, and within the constraints established at the beginning of the project.

9.0 Recommendations

Sputter far exceeded the expectations that the group had going into the project. Originally, the team only wanted to partly automate the cooking process; however, through the modularity of the design, they were able to iterate and build a complete start-to-finish automation system.

Although Sputter already surpassed what was expected from it, there are still many mechanical and software changes the team wants to make so Sputter can be improved, especially for its commercial viability.

9.1 Mechanical Design

Our data from verification already confirms the consistency of Sputter; however, it can always be better. The robot began to fail more frequently later in the demo. After roughly two hours of being run, there was a drastic change in the consistency of the flipping and plating performed by Sputter. When dissecting the issue after the demo, it was found that this failure was caused by the gear holding the spatula (see Figure 25). It had begun to warp under the weight and duration of being heated, which caused the axil to become loose and added slack to the spatula. To fix this, it is recommended to use a longer spatula shaft that is supported in at least two locations.

Another recommendation is that if longer cables could be sourced, then the Y-axis could be extended, allowing for larger and higher quantities of pancakes to be made at once. This would also allow for more complex designs to be printed as the robot would have a larger build volume to work with.

The pneumatic system was an extremely effective method of applying pressure on the batter; however, the regulator was quite poor at low pressures. Between runs, the regulator would have to be recalibrated and was very faulty in only allowing small amounts of air through, so to improve the extrusion flow rate, an upgraded low-pressure regulator would be of great benefit.

Finally, to reach a 100% success rate, the robot needs more active feedback on its location and the pancake's location. This can likely be done by incorporating some sort of vision and distance sensing so that the extruder not only knows its location but also the location of the pancake in case of errors, causing a shift in its location.

9.2 Software Recommendations.

If more development time were available, the software would be expanded to include more versatile PID exit conditions to allow curves to be drawn smoothly. For development into a consumer product, the software should make use of concurrent programming to ensure that motors and sensors are continually updated on a predictable timeframe. The program should also incorporate limit switches to provide a reliable homing routine. To enhance the user experience, an interface would be added to convert a 2D vector image into usable motor movements to allow the user to draw new shapes rather than relying on hard coded paths. Additionally, being able to access temperature information from the motors would improve safety by enabling the system to detect and respond to overheating conditions before damage occurs.

References

- [1] Biswas S, Alam SS, Sayem AH, Hossain M, Mithu SH, Akther S, "Breakfast skipping and associated factors: experience from students at Public University in Noakhali District," pp. 1-17, 2020.
- [2] Khan, M.S.I., Paul, T., Al Banna, M.H., "Skipping breakfast and its association with sociodemographic characteristics, night eating syndrome, and sleep quality among university students in Bangladesh," 2024.
- [3] Adolphus, K., Lawton, C. L., & Dye, L., "The effects of breakfast on behavior and academic performance in children and adolescents.," *Frontiers in Human Neuroscience*, 2013.
- [4] Michele Ferrara, Luigi De Gennaro, "How much sleep do we need?," *Sleep Medicine Reviews*, vol. 5, no. 2, 2001.
- [5] M. Olsen, L. Kane, F. Souan, A. Choudry, "2025 1A Final Project Idea Submission," 2025.
- [6] S. Magoo, "An Analysis of the Impact of Teaching Methods on Student Academic Performance," in *Engineering Education Conference*, Seattle, 2015.
- [7] IEEE, "IEEE Referencing: A Users Guide," Waterloo Publishing, Waterloo, 2017.

Appendix A – C++ Code

```
#pragma region VEXcode Generated Robot Configuration
// Make sure all required headers are included.
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <math.h>
#include <string.h>

#include "vex.h"

using namespace vex;

// Brain should be defined by default
brain Brain;

// START IQ MACROS
#define waitUntil(condition) \
do { \
    wait(5, msec); \
} while (!(condition))

#define repeat(iterations) \
for (int iterator = 0; iterator < iterations; iterator++)

// END IQ MACROS

// Robot configuration code.
inertial BrainInertial = inertial();
motor Motor12 = motor(PORT12, false);

// generating and setting random seed
void initializeRandomSeed(){
    wait(100,msec);
    double xAxis = BrainInertial.acceleration(xaxis) * 1000;
    double yAxis = BrainInertial.acceleration(yaxis) * 1000;
    double zAxis = BrainInertial.acceleration(zaxis) * 1000;
    // Combine these values into a single integer
    int seed = int(
        xAxis + yAxis + zAxis
    );
    // Set the seed
    srand(seed);
}

void vexcodeInit() {

    // Initializing random seed.
    initializeRandomSeed();
}

#pragma endregion VEXcode Generated Robot Configuration

//-----
//
// Module:      main.cpp
```

```

// Author:      Lennart, Aydin, Matthew, Felix
// Created:     Oct 19 2025
// Description: Pancake Making Robot - Sputter
//
//-----

// Include the IQ Library
#include "iq_cpp.h"

// Allows for easier use of the VEX Library
using namespace vex;

motor x(2, true);
motor y(9, false);
motor z(8, true);
motor flipper(7, false);
touchled TouchLED(8);
bumper BumperSensor(5);
distance Distance(0);
optical Optical3(3);

//constants to convert motor degrees into millimetres. Makes it easier to code autonomous
programs using "GCODE"
const double XDEGTOMM = 210.0/1000.0;
const double YDEGTOMM = 250.0/1500.0;
const double ZDEGTOMM = 150.0/820.0;

const double DEADBAND = 2.0; //deadband of 2mm for PID
const int KP = 2; //pid proportional constant

const int COOKTIME1 = 90;
const int COOKTIME2 = 90;

void waitFor(int millisec);

//if the program is waiting for the user to start the printing of the next pancake.
//used to control the exit program distance sensor.
bool pancakeWait = true;

//runs a while loop that consistently ticks the sensors until the motor has reached a desired
position.
void waitSteadyx() {
    while (abs(x.position(degrees) * XDEGTOMM - xTarget) > DEADBAND) {
        waitFor(10);
    }
}

void waitSteadyy() {
    while (abs(y.position(degrees) * YDEGTOMM - yTarget) > DEADBAND) {
        waitFor(10);
    }
}

void waitSteadyz() {
    while (abs(z.position(degrees) * ZDEGTOMM - zTarget) > DEADBAND) {
        waitFor(10);
    }
}

void waitSteadyflipper() {

```

```

    while (abs(flipper.position(degrees) - flipperTarget) > DEADBAND) {
        waitFor(10);
    }
}

//spins the filler motor to preset positions that will open or close the dispensing system
void DispenseOn() {
    flipperTarget = 90*6.5;
}

void DispenseOff() {
    flipperTarget = 0;
}

//checks if the emergency state has been triggered. includes a simple debouncing to account
for sensor errors
bool checkEmergency(bumper& B) {
    static int pressingCount = 0;

    if (B.pressing()) {
        pressingCount++;
    } else {
        pressingCount--;
    }

    if (pressingCount < 0) {
        pressingCount = 0;
    }

    if (pressingCount > 3) {
        return true;
    } else {
        return false;
    }
}

//spins the motor in reverse until it detects that the motor has stopped and the
//motor current goes above a certain threshold stops the motor and resets the position
void Zero(motor& m) {
    m.spin(reverse, 95, percent);

    while (m.velocity(percent) > 10 || m.current(amp) < 0.5) {
        wait(10,msec);
    }

    m.stop();
    m.setPosition(0,degrees);
}

//homes the motors one by one. Moves the motor into an acceptable area where the subsequent
motor can zero
void ZeroAll() {
    Zero(z);
    zTarget = 90;
    waitSteadyz();

    Zero(x);
    xTarget = 100;
    waitSteadyx();
}

```



```

Zero(y);
yTarget = 50;
waitSteady();

//home
xTarget = 5;
yTarget = 150;
zTarget = 10;
flipperTarget = 0;
waitFor(1000);
}

//main tick function. updates all sensors on a constant 10ms timer
void waitFor(int millisec) {
    //loop enough times to elapse the desired wait time
    for (int i = 0; i < millisec/10; i++) {

        //move x motor proportional to error
        double xError = xTarget - x.position(degrees) * XDEGTOMM;

        if (xError > xMaxSpeed) {
            xError = xMaxSpeed;
        } else if (xError < -xMaxSpeed) {
            xError = -xMaxSpeed;
        }

        if (abs(xError) > DEADBAND) {
            x.spin(forward, xError * KP, percent);
        } else {
            x.stop();
        }

        //move y motor proportional to error
        double yError = yTarget - y.position(degrees) * YDEGTOMM;

        if (yError > yMaxSpeed) {
            yError = yMaxSpeed;
        } else if (yError < -yMaxSpeed) {
            yError = -yMaxSpeed;
        }

        if (abs(yError) > DEADBAND) {
            y.spin(forward, yError * KP, percent);
        } else {
            y.stop();
        }

        //move z motor proportional to error
        double zError = zTarget - z.position(degrees) * ZDEGTOMM;

        if (zError > zMaxSpeed) {
            zError = zMaxSpeed;
        } else if (zError < -zMaxSpeed) {
            zError = -zMaxSpeed;
        }

        if (abs(zError) > DEADBAND) {
            z.spin(forward, zError * KP, percent);
        } else {
            z.stop();
        }
    }
}

```

```

}

//move flipper motor proportional to error
double flipperError = flipperTarget - flipper.position(degrees);

if (flipperError > 50) {
    flipperError = 50;
} else if (flipperError < -50) {
    flipperError = -50;
}

if (abs(flipperError) > DEADBAND) {
    flipper.spin(forward, flipperError, percent);
} else {
    flipper.stop();
}

//check for emergency stop. if pressed, reverse z axis to remove from hotplate
if (checkEmergency(BumperSensor)) {
    Brain.Screen.setFont(vex::fontType::mono20);
    Brain.Screen.setFillColors(red);
    Brain.Screen.setPenColor(red);
    Brain.Screen.drawRect(1,1,200,200);
    Brain.Screen.setCursor(2, 4);
    Brain.Screen.setPenColor(white);
    Brain.Screen.print("Emergency");
    Brain.Screen.setCursor(4, 6);
    Brain.Screen.print("Exit");

    //user feedback letting the user know there was an alarm
    Brain.playSound(alarm);

    x.stop();
    y.stop();
    z.stop();
    flipper.stop();

    DispenseOff();
    Zero(z);
    waitSteadyflipper();

    z.spin(reverse, 5, percent);

    //keep moving the z motor away from the hotplate. alarm for 20 seconds before
    terminating the program
    for (int i = 0; i < 100; i++) {

        Brain.playSound(alarm);
        wait(200 ,msec);
    }
    Brain.programStop();
}

//if something touches the distance sensor while it is idling, terminate program
//user controlled ending to the program
if(Distance.objectDistance(mm) < 2 && pancakeWait) {
    Brain.programStop();
}

```

```

    wait(10, msec);
}
}

//wait for a certain amount of seconds and display a dynamic progress bar
void WaitCook(int sec) {
    for (int i = sec; i != 0; i--) {
        Brain.Screen.clearScreen();
        Brain.Screen.setPenColor(blue);
        Brain.Screen.drawRect(0, 0, 160, 108);
        Brain.Screen.setPenColor(white);
        Brain.Screen.setPenColor(vex::color::white);
        Brain.Screen.setCursor(3, 4);
        Brain.Screen.print("Time until flip:");
        Brain.Screen.setCursor(4, 6);
        Brain.Screen.print("%d seconds", i-1);

        //draw dynamic progress bar
        int width = (int)((sec - i + 1) * 138.0 / sec);

        if (i != 1) {
            Brain.Screen.setPenColor(white);
            Brain.Screen.setFill-color(black);
            Brain.Screen.drawRect(10, 90, 140, 10, white);
            Brain.Screen.drawRect(12, 92, width, 6);
        } else {
            Brain.Screen.setPenColor(green);
            Brain.Screen.setFill-color(green);
            Brain.Screen.drawRect(10, 90, 140, 10);
        }

        waitfor(1000);
    }

    Brain.Screen.setPenColor(white);
    Brain.Screen.setFill-color(black);
    Brain.Screen.setFont(mono15);
}

//Gcode converted to draw shapes with our robot extruded pancake batter
void drawSpade() {
    xTarget = 176.99;
    yTarget = 194.13;
    waitfor(1000);

    zTarget = 120;
    waitfor(600);

    DispenseOn();
    waitSteadyflipper();

    xTarget = 190.72;
    yTarget = 178.33;
    waitfor(157);

    xTarget = 200.54;
    yTarget = 164.73;
    waitfor(136);

    xTarget = 206.87;

```

```

yTarget = 153.94;
waitFor(106);

xTarget = 212.04;
yTarget = 142.71;
waitFor(112);

xTarget = 215.93;
yTarget = 131.07;
waitFor(116);

xTarget = 218.35;
yTarget = 119.08;
waitFor(119);

xTarget = 219.07;
yTarget = 110.74;
waitFor(83);

xTarget = 218.66;
yTarget = 98.00;
waitFor(127);

xTarget = 217.79;
yTarget = 92.30;
waitFor(57);

xTarget = 214.50;
yTarget = 81.86;
waitFor(104);

xTarget = 212.19;
yTarget = 77.12;
waitFor(47);

xTarget = 206.31;
yTarget = 68.67;
waitFor(84);

xTarget = 202.79;
yTarget = 64.97;
waitFor(37);

xTarget = 194.88;
yTarget = 58.69;
waitFor(79);

xTarget = 190.52;
yTarget = 56.12;
waitFor(43);

xTarget = 181.16;
yTarget = 52.24;
waitFor(93);

xTarget = 176.24;
yTarget = 50.93;
waitFor(49);

xTarget = 166.17;

```

```
yTarget = 49.60;
waitFor(100);

xTarget = 160.98;
yTarget = 49.61;
waitFor(51);

xTarget = 150.50;
yTarget = 51.05;
waitFor(104);

xTarget = 145.36;
yTarget = 52.48;
waitFor(51);

xTarget = 134.86;
yTarget = 56.89;
waitFor(105);

xTarget = 130.07;
yTarget = 59.85;
waitFor(47);

xTarget = 120.84;
yTarget = 67.39;
waitFor(92);

xTarget = 116.66;
yTarget = 71.99;
waitFor(45);

xTarget = 120.18;
yTarget = 52.98;
waitFor(190);

xTarget = 128.49;
yTarget = 25.51;
waitFor(274);

xTarget = 138.69;
yTarget = 0.00;
waitFor(255);

xTarget = 80.69;
waitFor(580);

xTarget = 82.74;
yTarget = 6.59;
waitFor(65);

xTarget = 97.66;
yTarget = 47.01;
waitFor(404);

xTarget = 99.64;
yTarget = 56.57;
waitFor(95);

xTarget = 100.41;
yTarget = 63.17;
```

```

    waitFor(66);

    xTarget = 87.72;
    yTarget = 58.78;
    waitFor(126);

    xTarget = 82.73;
    yTarget = 56.03;
    waitFor(49);

    DispenseOff();
    waitFor(10000);
}

void drawSquare() {
    xMaxSpeed = 10;
    yMaxSpeed = 10;
    zMaxSpeed = 10;

    zTarget = 120;
    waitSteadyz();

    yTarget = 200;
    xTarget = 180;
    waitSteadyy();

    DispenseOn();
    waitSteadyflipper();

    yTarget = 100;
    waitSteadyy();

    xTarget = 80;
    waitSteadyx();

    yTarget = 200;
    waitSteadyy();

    xTarget = 180;
    waitSteadyx();

    yTarget = 100;
    waitSteadyy();

    DispenseOff();
    waitSteadyflipper();

    xMaxSpeed = 100;
    yMaxSpeed = 100;
    zMaxSpeed = 100;

}

void drawStar() {
    xMaxSpeed = 20;

```

```

yMaxSpeed = 20;
zMaxSpeed = 20;

zTarget = 110;
waitSteadyz();

xTarget = 109.99;
yTarget = 155.17; // Was 125.17
waitSteadyx();
waitSteadyy();

DispenseOn();
waitSteadyflipper();

xTarget = 11.41;
yTarget = 86.55; // Was 56.55
waitSteadyx();
waitSteadyy();

xTarget = 50.91;
yTarget = 199.73; // Was 169.73
waitSteadyx();
waitSteadyy();

xTarget = 85.79;
yTarget = 84.83; // Was 54.83
waitSteadyx();
waitSteadyy();

xTarget = 90.01;
yTarget = 157.41; // Was 127.41
waitSteadyx();
waitSteadyy();

xTarget = 109.99;
yTarget = 155.17; // Was 125.17
waitSteadyx();
waitSteadyy();

xTarget = 109.99;
yTarget = 155.17; // Was 125.17
waitSteadyx();
waitSteadyy();

xTarget = 11.41;
yTarget = 86.55; // Was 56.55
waitSteadyx();
waitSteadyy();

xTarget = 50.91;
yTarget = 199.73; // Was 169.73
waitSteadyx();
waitSteadyy();

xTarget = 85.79;
yTarget = 84.83; // Was 54.83
waitSteadyx();
waitSteadyy();

xTarget = 90.01;

```

```

yTarget = 157.41; // Was 127.41
waitSteadyx();
waitSteadyy();

xTarget = 109.99;
yTarget = 155.17; // Was 125.17
waitSteadyx();
waitSteadyy();

DispenseOff();
waitSteadyflipper();

xMaxSpeed = 100;
yMaxSpeed = 100;
zMaxSpeed = 100;
}

//dispenses batter for a certain amount of time, checks if there is sufficient batter and
//closes
//the extrusion if the batter ever falls below a certain level
bool dispenseBatter(double dispenseTime){

    flipper.setVelocity(60,percent);

    double tempPosition = abs(flipper.position(degrees));
    double positionOn = tempPosition + 60;
    double startTime = Brain.Timer.value();

    flipper.spin(forward);

    while (abs(flipper.position(degrees)) < positionOn){}

    flipper.stop();

    while ((Optical3.hue() > 30 && Optical3.hue() < 50) && Brain.Timer.value() < (startTime +
dispenseTime)){

        flipper.spin(reverse);

        while (abs(flipper.position(degrees)) > tempPosition){}
        flipper.stop();

        if (Optical3.hue() > 30 && Optical3.hue() < 50){
            return true;
        }
        else{
            return false;
        }
    }
}

int main() {
    // Initializing Robot Configuration. DO NOT REMOVE!
    vexcodeInit();
    // Begin project code

    x.setStopping(brake);
    y.setStopping(brake);
    z.setStopping(brake);
    flipper.setStopping(brake);
}

```



```

int numPancakes = 0;

Brain.Screen.setPenWidth(1);

bool first = true;

while (true) {

    Brain.Screen.setFont(vex::fontType::mono12);
    TouchLED.setColor(red);
    Brain.Screen.clearScreen();
    Brain.Screen.drawRect(0, 0, 158, 106);
    Brain.Screen.setPenColor(white);
    Brain.Screen.setCursor(3, 3);
    Brain.Screen.print("Touch sensor to");
    Brain.Screen.setCursor(4, 3);
    Brain.Screen.print("start next pancake");

    while (!TouchLED.pressing()) {
        waitFor(10);
    }

    pancakeWait = false;

    Brain.Screen.setFont(vex::fontType::mono20);
    Brain.Screen.setPenColor(vex::color::white);

    if (first) {
        Brain.Screen.setPenColor(red);
        Brain.Screen.clearScreen();
        Brain.Screen.drawRect(0, 0, 158, 106);
        Brain.Screen.setPenColor(white);
        Brain.Screen.setCursor(3, 2);
        Brain.Screen.print("Zeroing Motors");
    } else {
        Brain.Screen.setPenColor(green);
    }

    Brain.Screen.setFont(vex::fontType::mono12);

    ZeroAll();

    if (first) {
        Brain.Screen.setPenColor(red);
    } else {
        Brain.Screen.setPenColor(green);
    }

    Brain.playSound(headlightsOn);

    //extrude pancake
    xTarget = 210;
    waitSteadyX();

    yTarget = 190;
    waitSteadyY();

    zTarget = 105;
    waitSteadyZ();
}

```

```

DispenseOn();
waitSteadyflipper();

waitFor(2500);

DispenseOff();
waitSteadyflipper();

xTarget = 5;
yTarget = 150;
zTarget = 10;
flipperTarget = 0;
waitFor(1000);

TouchLED.setColor(blue);
//wait for first pancake side to cook
WaitCook(COOKTIME1);

Brain.Screen.clearScreen();
Brain.Screen.setPenColor(blue);
Brain.Screen.drawRectangle(0, 0, 158, 106);
Brain.Screen.setPenColor(white);
Brain.Screen.setCursor(3, 3);
Brain.Screen.print("Flipping Pancake");

//start moving
yTarget = 140;
waitSteadyy();

zTarget = 155;
waitSteadyz();

xTarget = 130;
waitSteadyx();

xTarget = 110;
yTarget = 160;
zTarget = 110;
waitSteadyz();

flipperTarget = 180;
waitSteadyflipper();
waitFor(500);

//home
xTarget = 5;
yTarget = 20;
zTarget = 5;
flipperTarget = -4;
waitFor(1000);

//wait for second pancake side to cook
WaitCook(COOKTIME2);

Brain.Screen.clearScreen();
Brain.Screen.setPenColor(blue);
Brain.Screen.drawRectangle(0, 0, 158, 106);
Brain.Screen.setPenColor(white);
Brain.Screen.setCursor(3, 3);
Brain.Screen.print("Plating Pancake");

```

```

    yTarget = 150;
    waitSteady();

    zTarget = 155;
    waitSteadyz();

    xTarget = 130;
    waitSteadyx();

    xTarget = 120;
    yTarget = 140;
    zTarget = 110;
    waitSteadyz();

    xTarget = 200;
    waitSteadyx();

    flipperTarget = 180;
    waitSteadyflipper();
    waitFor(500);

    //home
    xTarget = 10;
    yTarget = 20;
    zTarget = 5;
    flipperTarget = -4;
    waitFor(1000);

    Brain.Screen.setPenColor(green);
    Brain.Screen.drawRectangle(0, 0, 158, 106);
    Brain.Screen.setPenColor(white);
    Brain.Screen.setCursor(3, 3);
    Brain.Screen.print("Pancake Finished!");

    Brain.playSound(tada);
    TouchLED.setColor(green);
    first = false;

    numPancakes++;
    pancakeWait = true;

    waitFor(1000);
}
}

```